Propuesta de Arquitectura de Integración

Modernización de Sistemas Bancarios
Multi-Core · Event-Driven · BIAN

Versión 1.3 — Julio 2026
Entregable: Context / Container / Component / Saga (diagramas visuales) + diseño técnico

Propuesta de Arquitectura de Integración

Modernización de Sistemas Bancarios — Enfoque Multi-Core, Event-Driven y alineado a BIAN

Versión: 1.3
Fecha: Julio 2026
Tipo de documento: Entregable de arquitectura de integración

Tabla de contenidos

1. Resumen ejecutivo
2. Principios de diseño y alineación BIAN
3. Diagramas C4
4. Patrones de integración y tecnologías
5. Seguridad, cumplimiento y protección de datos
6. Alta disponibilidad y recuperación ante desastres
7. Estrategia de integración multi-core
8. Gestión de identidades y accesos (IAM)
9. Estrategia de APIs internas y externas
10. Gobernanza de APIs y microservicios
11. Plan de migración gradual
12. Observabilidad y trazabilidad
13. Anexos

1. Resumen ejecutivo

Se propone una arquitectura de integración **híbrida event-driven + API-first**, orientada a modernizar un banco tradicional sin big-bang. El diseño:

- Convive el **Core Tradicional (SoR legacy)** con un **Core Digital (SoR moderno)** mediante un **Multi-Core Orchestration Layer** y un **Anti-Corruption Layer (ACL)**.
- Expone canales (web/móvil), pagos, Open Finance, riesgos y fraude a través de un **API Management Plane** y un **Event Backbone** (Kafka / Azure Event Hubs compatible).
- Se alinea a **BIAN Service Domains** para delimitar bounded contexts, ownership de datos y contratos semánticos.

- Privilegia **cohesión alta / acoplamiento bajo**, contratos explícitos, evolución independiente y **sin bases de datos compartidas** entre microservicios.
- Usa cloud **Azure** como referencia de infraestructura (portable a AWS con equivalencias documentadas), con API Manager (Azure API Management), Key Vault, AKS y Event Hubs / Kafka.

Resultado esperado: baja latencia en caminos críticos (consulta de saldo, autorización de pago), consistencia eventual controlada en procesos multi-core, cumplimiento regulatorio (PCI-DSS, Open Finance, protección de datos personales) y un camino de migración Strangler Fig medible.

1.1 Caso de uso unificado (recorrido extremo a extremo)

Escenario de referencia usado en C4, sagas, IAM y observabilidad:

1. **Cliente** autentica en banca móvil (CIAM → token con scopes `payments.write`).
2. **Mobile BFF** crea `POST /internal/v1/payment-orders` vía APIM (OAuth2 + `Idempotency-Key` + `correlationId`).
3. **Payment Order** valida contrato BIAN; **Payment Guard** consulta límites (Risk cache).
4. **Fraud Evaluation** síncrono (≤ 80 ms p99); si `ALLOW`, continúa; si `CHALLENGE`/`DENY`, corta el camino.
5. **Saga cross-core** (origen Legacy → destino Digital): Debit Legacy → Credit Digital → Confirm (ver §3.5 / §7.2).
6. **Outbox** publica `payment.execution.completed.v1` + `account.balance.changed.v1` al Event Backbone.
7. Consumidores: Notificaciones, Account Read Model, Audit WORM; Open Finance solo ve datos bajo consentimiento activo.

Este flujo ancla latencia (§3.4), HA/DR de sagas in-flight (§6.5) y SLOs (§12.4).

---

2. Principios de diseño y alineación BIAN

2.1 Principios arquitectónicos aplicados

Principio	Cómo se refleja en el diseño
Single Responsibility	Cada microservicio posee un único dominio BIAN (p. ej. `PaymentExecution` no calcula riesgo).
Interface Segregation	APIs internas granulares vs. APIs externas agregadas (Open Finance / Channel BFF).
Dependency Inversion	Canales dependen de puertos (interfaces) del Integration Layer; adaptadores concretos hacia cores.
Separation of Concerns	Gateway solo enruta/autentica; orquestación en Multi-Core Orchestrator; lógica de negocio en domain services.
Clean Architecture	Dominio → Application → Adapters (inbound API/events, outbound cores/pagos).
Database per Service	Cada bounded context tiene su propio store; integración solo por API/eventos.

2.2 Dominios BIAN seleccionados (matriz de dominios)

Dominio BIAN	Responsabilidad	System of Record	Consumidores principales
Party Reference Data Management	Identidad del cliente, KYC referencial	Core Digital + MDM Cliente	Canales, Open Finance, Fraude
Current Account	Cuentas a la vista, saldos, movimientos	Core Tradicional (fase 1) → Core Digital (fase N)	Canales, Pagos, Riesgos

Payment Order / Payment Execution	Órdenes e instrucción de pago	Plataforma de Pagos + Core destino	Canales, Open Finance
Card Transaction	Autorizaciones y clearing tarjetas	Core Tradicional / Switch	Fraude, Canales
Fraud Evaluation	Scoring y decisión de fraude en tiempo real	Sistema Fraude	Payment Execution, Card
Market Risk / Credit Risk	Evaluación y límites	Sistema Riesgos	Onboarding, Crédito, Pagos alto monto
Product Directory	Catálogo de productos	Core Digital	Canales, Open Finance
Channel Activity History	Auditoría de interacción canal	Event Store / Analytics	Cumplimiento, CX
Servicing Order	Casos operativos / backoffice	BPM / Case Mgmt	Operaciones

> **Nota BIAN:** Esta matriz lista los dominios del diseño. El **landscape** (§2.2.1) muestra cómo se relacionan en un pago; el **mapeo** (§2.2.2) fija API / evento / SoR; el **Control Record** (§2.2.3) ancla ownership de dinero. El Integration Layer traduce y orquesta; no mezcla dominios ni comparte DB.

2.2.1 Landscape BIAN operativo (Service Domains + relaciones)

Un **Service Domain BIAN**, en este diseño, es un **bounded context con ownership**: un equipo dueño, un System of Record (SoR), y contratos propios (API + eventos). No es un catálogo genérico BIAN: solo aparecen los dominios que la plataforma usa. El Integration Layer **orquesta y traduce**; no mezcla datos entre dominios ni comparte bases.

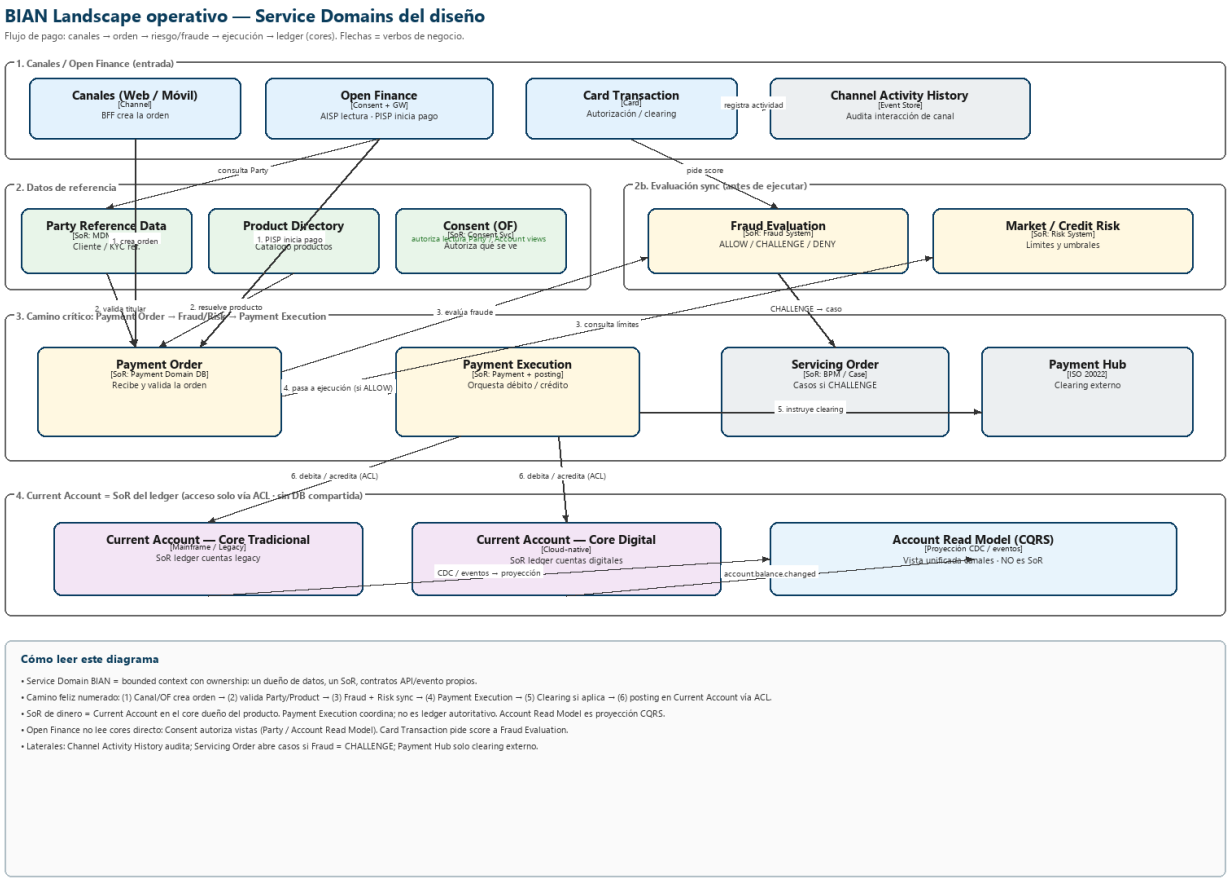


Figura 0. Landscape BIAN operativo — dominios, flujo de pago y SoR

Cómo se lee (de arriba hacia abajo):

Capa	Dominios	Rol en el diseño
1. Entrada	Canales (Web/Móvil), Open Finance (Consent + AISP/PISP), Card Transaction	Origen de la interacción. El canal/PISP **crea** la orden; la tarjeta **pide score** a fraude.
2. Referencia	Party Reference Data, Product Directory, Consent	Datos maestros / autorización. Party y Product **validan** titular y producto; Consent **autoriza** qué ve Open Finance.
3. Camino de pago	Payment Order → Fraud Evaluation + Market/Credit Risk → Payment Execution	Orden valida; Fraud/Risk deciden en sync; Execution **orquesta** el posting. Servicing Order abre casos si `CHALLENGE`.
4. Ledger (SoR)	Current Account (Core Tradicional y/o Core Digital)	Dueño autoritativo del saldo. Execution debita/acredita **vía ACL** (puerto `CoreAccountPort`), nunca contra la DB del core.

Quién llama a quién en un pago (verbos de negocio):

1. Canal / Open Finance (PISP) → **crea orden** → `Payment Order`
2. `Payment Order` → **valida titular / producto** → Party, Product Directory
3. `Payment Order` / Guard → **consulta límites** → Market/Credit Risk (cache)
4. Camino crítico → **evalúa fraude** → Fraud Evaluation (`ALLOW` | `CHALLENGE` | `DENY`)
5. Si `ALLOW` → `Payment Execution` → **debita / acredita** → Current Account (core dueño, vía ACL)
6. Si clearing externo → Execution → **instruye clearing** → Payment Hub
7. Tras posting → cores **publican / proyectan** → Account Read Model (CQRS; **no** es SoR)
8. Lateral: canales/OF → **registran actividad** → Channel Activity History; Fraud `CHALLENGE` → **abre caso** → Servicing Order

Quién es SoR de qué (resumen):

Qué	SoR	Quién NO es SoR
Identidad / KYC referencial	Party MS / MDM	Canales, Fraude (solo consumen)
Catálogo de productos	Core Digital / Product MS	Payment Domain
Consentimiento Open Finance	Consent Service	APIM (solo aplica policy)
Orden e instrucción de pago	Payment Domain DB (+ estado de ejecución)	Orchestrator (coordina, no posee el dinero)
Decisión de fraude / límites	Fraud System / Risk System	Payment Execution
Saldo y asientos (ledger)	**Current Account en el core dueño**	Payment DB, Read Model, Orchestrator

Mismo flujo en una línea (lectura rápida):
Canal/OF PISP crea orden → Payment Order valida Party/Product y consulta Risk → Fraud Evaluation decide → Payment Execution debita/acredita Current Account (vía ACL) y, si aplica, instruye Payment Hub → eventos/CDC alimentan el Account Read Model (proyección, no SoR).

La Figura 0 (PNG) es la vista visual de este landscape. Detalle de contratos API/evento por dominio: §2.2.2. Ownership de dinero vs orquestación: §2.2.3.

2.2.2 Mapeo BIAN SD → API resource → Event type → SoR

Contratos canónicos de los mismos dominios del landscape (§2.2.1). Cada fila = un Service Domain con su API, evento y SoR.

BIAN Service Domain	API resource (canónico)	Event type	System of Record
Party Reference Data Management	`/internal/v1/parties/{id}`	`party.customer.updated.v1`	Party MS / MDM
Current Account	`/internal/v1/accounts/{id}`	`account.balance.changed.v1`	Core dueño del producto (Legacy o Digital)
Payment Order	`/internal/v1/payment-orders`	`payment.order.created.v1`	Payment Domain DB
Payment Execution	`/internal/v1/payment-executions/{id}`	`payment.execution.completed.v1` / `.failed.v1`	Payment Domain + posting en Core destino
Fraud Evaluation	`/internal/v1/fraud/decisions`	`fraud.decision.made.v1`	Fraud System
Credit / Market Risk	`/internal/v1/risk/limits`	`risk.limit.updated.v1`	Risk System
Product Directory	`/internal/v1/products`	`product.catalog.updated.v1`	Core Digital / Product MS
Channel Activity History	(solo eventos)	`channel.activity.recorded.v1`	Event Store / Analytics
Consent (OF)	`/open-finance/v1/consents`	`consent.granted.v1` / `.revoked.v1`	Consent Service

2.2.3 Control Record BIAN vs payload canónico (ejemplo)

Ownership explícito: `Payment Order` / `Payment Execution` poseen el ciclo de vida de la orden e instrucción. **Current Account** (ledger del core dueño) posee saldos y asientos. El Orchestrator **no** es SoR de dinero: solo coordina; el posting autoritativo ocurre en el core vía ACL. Un fallo de saga revierte postings con `ReverseDebit` / `ReverseCredit`; no se “ajusta” saldo desde Payment DB.

Aspecto	Control Record BIAN (conceptual)	Payload canónico del banco
Identidad	`PaymentExecution` CR id	`executionId` / `orderId`
Partes	Debtor / Creditor party refs	`debtorAccountId`, `creditorAccountId` (+ `customerId` en Party)
Importe	Amount + currency	`amount.value`, `amount.currency`
Estado	Behavior qualifier (Initiated…Completed)	`status`: `RECEIVED` → `FRAUD_CHECKED` → `DEBITED` → `CREDITED` → `COMPLETED` | `COMPENSATING` | `FAILED`
SoR path	N/A (extensión local)	`sorPath`: `["LEGACY","DIGITAL"]`
Evento	CR serializado / mensaje de dominio	CloudEvents `type=payment.execution.completed.v1` + `data` canónico

---

3. Diagramas C4

3.1 Nivel 1 — Context Diagram

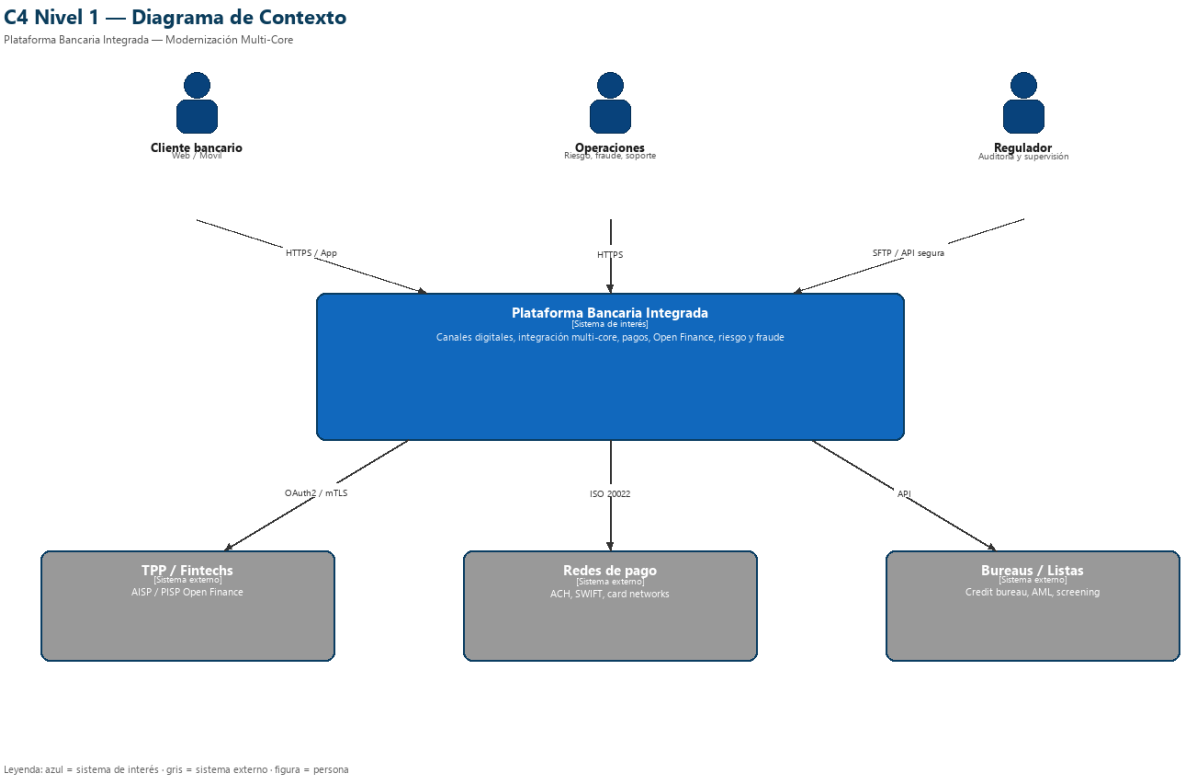


Figura 1. Context — Plataforma Bancaria Integrada

Lectura del contexto: el banco es el sistema de interés. Los actores externos no conocen la dualidad de cores; solo interactúan con la plataforma integrada.

3.2 Nivel 2 — Container Diagram

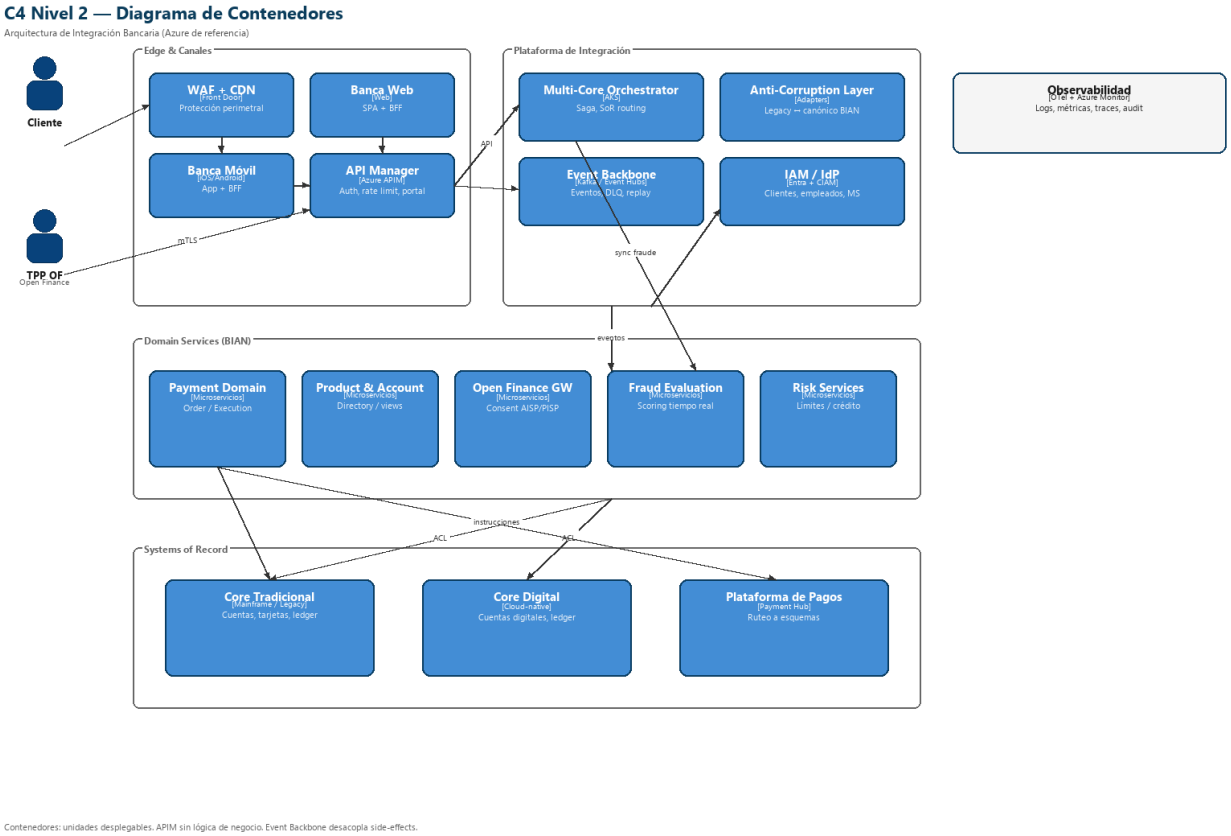


Figura 2. Container — Arquitectura de Integración

Decisiones clave del contenedor:

1. **API Manager** no contiene lógica de negocio (solo seguridad, throttling, versionado, mediación ligera).
2. **Multi-Core Orchestrator** decide *qué SoR* atiende cada operación y ejecuta sagas.
3. **ACL** aísla modelos legacy del modelo canónico BIAN.
4. **Event Backbone** desacopla fraude, riesgo, notificaciones, analytics y proyección de lecturas.

3.3 Nivel 3 — Component Diagram (Multi-Core Orchestrator + Payment)

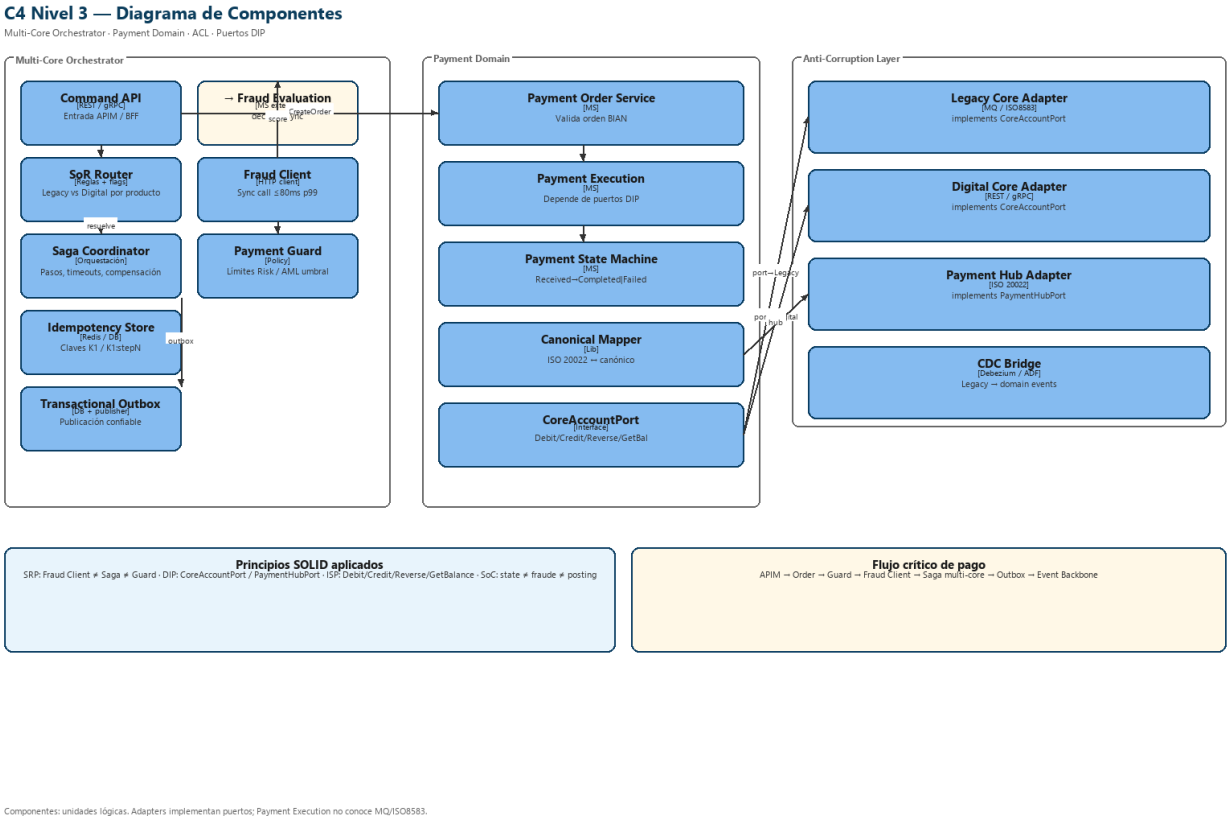


Figura 3. Component — Orchestrator, Payment y ACL

Componentes destacados (además de Router / Saga / Adapters):

Componente	Responsabilidad	Principio
Fraud Client	Llamada síncrona a Fraud Evaluation; timeout estricto; no decide reglas de fraude	SRP / SoC
Payment Guard	Pre-checks de límites (Risk cache), listas AML umbral, horario/canal	SRP
CoreAccountPort	Puerto DIP: `DebitAccount`, `CreditAccount`, `GetBalance`, `ReverseDebit`, `ReverseCredit`	DIP + ISP
PaymentHubPort	Puerto DIP: `InstructPayment`, `GetClearingStatus`	DIP
Legacy/Digital Adapters	Implementan `CoreAccountPort` traduciendo a MQ/ISO8583 o REST	ACL

SOLID en componentes:

- **SRP:** `SoR Router` solo enruta; `Saga Coordinator` solo coordina; adapters solo traducen; `Fraud Client` solo invoca fraude.
- **DIP:** `Payment Execution` depende de interfaces `CoreAccountPort` / `PaymentHubPort`, implementadas por adapters.
- **ISP:** puertos separados `DebitAccount`, `CreditAccount`, `GetBalance` — no un “God adapter”.
- **SoC:** state machine de pago ≠ evaluación de fraude ≠ posting contable.

3.4 Vista lógica de flujo crítico (Transferencia / Pago) + budget de latencia

Camino de escritura (pago / transferencia):

1. Canal / BFF
2. APIM (OAuth2 + mTLS + rate limit)
3. Payment Order
4. Payment Guard (límites / pre-checks)
5. Fraud Evaluation síncrono (objetivo: no más de 80 ms p99)
6. Multi-Core Orchestrator / Saga, que ejecuta:
   - débito en el SoR de origen (Legacy o Digital)
   - crédito en el SoR de destino (Legacy o Digital)
   - instrucción al Payment Hub si hay clearing externo
7. Outbox
8. Event Backbone
9. Consumidores: Notificaciones, Riesgos, Ledger Projection, Audit

Camino de consulta de saldo (baja latencia):
BFF → APIM → Account Read Model (proyección CQRS) → fallback al SoR solo si hay cache miss o invalidación.

Budget de latencia por hop (pago cross-core, objetivo p99 en región primaria):

Hop	Componente	Budget p99
1	APIM (auth + policy)	no más de 25 ms
2	Payment Order + Payment Guard	no más de 40 ms
3	Fraud Evaluation (sync)	no más de 80 ms
4	Saga step Debit (Legacy ACL)	no más de 200 ms
5	Saga step Credit (Digital ACL)	no más de 120 ms
6	Outbox commit (local TX)	no más de 15 ms
Total síncrono al cliente	(hasta accept / `ACCEPTED_FOR_EXECUTION`)	**no más de 350 ms**
Async	Publicación eventos + notificaciones	fuera del path UX

Si Fraud supera 80 ms → fail-open controlado solo para montos bajo umbral (configurable) o fail-closed con `503`/`408` según política de producto. Consulta saldo (read model): budget extremo a extremo **no más de 80 ms p99**.

3.5 Secuencia saga cross-core (Legacy → Digital)

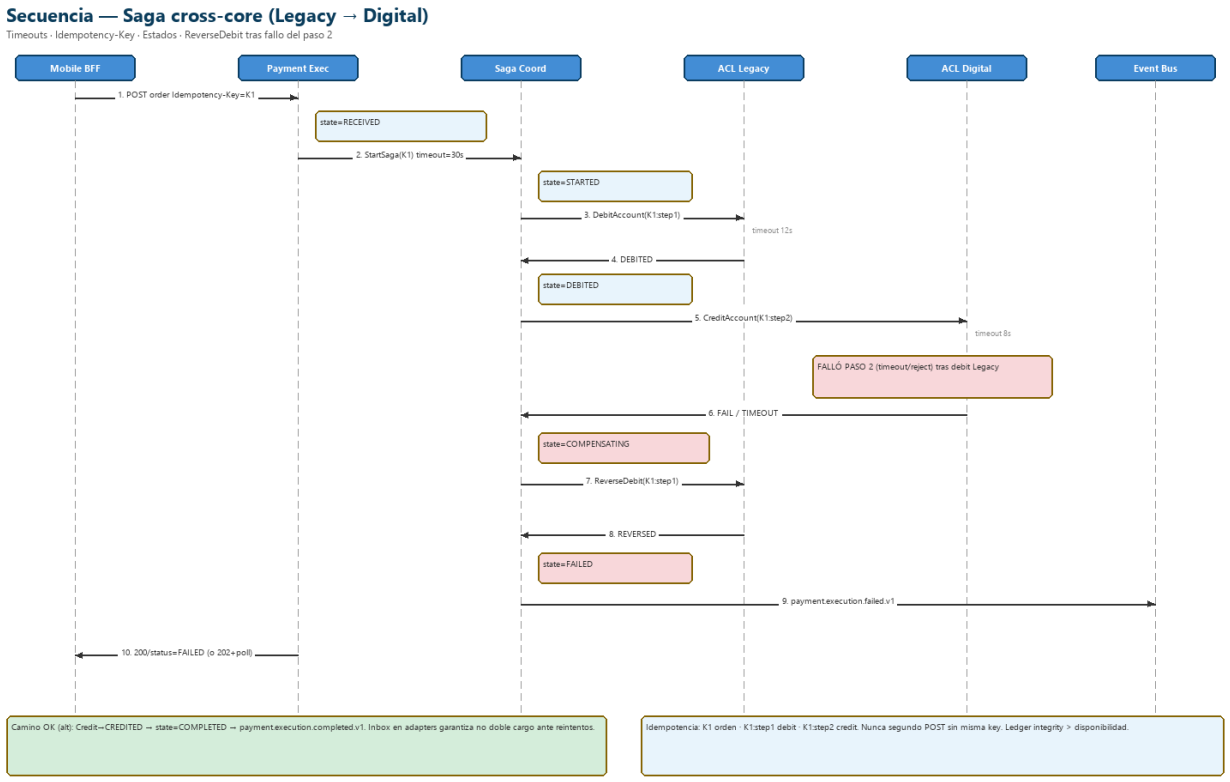


Figura 5. Saga cross-core — Debit Legacy → Credit Digital con compensación

La Figura 5 muestra el camino feliz (Debit Legacy → Credit Digital → evento completado) y el camino de fallo del paso 2 (compensación con `ReverseDebit` y evento `payment.execution.failed.v1`).

Contratos de la saga:

Elemento	Regla
Idempotency	Clave `K1` en orden; subclaves `K1:step1` / `K1:step2` en cada posting; Inbox en cores/adapters
Timeouts	Saga global 30 s; paso Credit 8 s; Debit Legacy 12 s
Estados	`STARTED` → `DEBITED` → `COMPLETED` o `COMPENSATING` → `FAILED`
Compensación	Tras débito Legacy exitoso y fallo de crédito Digital: **ReverseDebit**; si hubiera crédito previo fallido en reverse path: **ReverseCredit**
Cliente	Respuesta síncrona `202`/`200` con `status=ACCEPTED` o resultado final según producto; siempre evento terminal auditado

---

4. Patrones de integración y tecnologías

4.1 Patrones aplicados (dónde y por qué)

Patrón	Dónde	Por qué
API Gateway / API Management	Perímetro canales y Open Finance	Autenticación centralizada, cuotas, versionado, developer portal; sin lógica de negocio.
Backend for Frontend (BFF)	Banca web y móvil	Agrega APIs internas a payloads UX; evita chatty APIs desde el dispositivo.

Anti-Corruption Layer	Entre cores legacy/digital y dominio canónico	Evita filtrar modelos COBOL/ISO8583 al resto del banco.
Strangler Fig	Migración producto a producto hacia Core Digital	Reduce riesgo operativo vs. cutover único.
Saga (orquestada)	Transferencias multi-core / pagos multi-paso	Consistencia eventual con compensación explícita (`ReverseDebit`).
Transactional Outbox + Inbox	Domain services productores/consumidores	Entrega confiable sin 2PC distribuido.
CQRS + Read Models	Saldos y movimientos para canales	Baja latencia de lectura sin saturar cores.
Event-Driven Choreography	Notificaciones, analytics, sync de proyecciones	Desacopla side-effects; escala consumidores independientemente.
Circuit Breaker + Bulkhead	Llamadas a Core Tradicional y bureaus	Aísla latencia/fallos del legacy.
Idempotent Receiver	Pagos, postings, webhooks Open Finance	Reintentos seguros sin doble cargo.
Claim Check	Eventos con adjuntos grandes (extractos)	Mantiene payloads livianos en el bus.
Database per Service	Todos los MS de dominio	Ownership claro; evita coupling por schema.

4.2 Stack tecnológico de referencia (Azure, justificado)

Capacidad	Tecnología	Justificación
Cómputo MS	**AKS**	Portabilidad, densidad, service mesh opcional (Istio/Linkerd).
API Manager	**Azure API Management**	Políticas, OAuth, product plans Open Finance, analytics.
Eventos	**Kafka (Confluent/MSK equiv.) o Event Hubs Kafka protocol**	Orden por partición (accountId), replay, ecosistema maduro.
Identidad	**Microsoft Entra ID + External ID (CIAM)**	Empleados + clientes + workload identities.
Secretos	**Key Vault + Managed Identity**	Sin secretos en código; rotación.
Datos	**PostgreSQL/Cosmos (por servicio)**, Redis	Aislamiento + cache/idempotency.
CDC	**Debezium / Azure Data Factory + Event Hubs**	Extrae cambios del core legacy sin batch nocturno único.
Observabilidad	**OpenTelemetry** → **Azure Monitor / Grafana / Jaeger**	Estándar vendor-neutral.
IaC	**Terraform / Bicep**	Reproducibilidad DR y ambientes.

> Equivalencias AWS: APIM→API Gateway+AWS Marketplace portal, AKS→EKS, Event Hubs→MSK, Key Vault→Secrets Manager, Entra→Cognito+IAM Identity Center.

4.3 Diseño de eventos (estándar de industria)

Convenciones:

- Nombre: `domain.entity.action.vN` (ej. `payment.execution.completed.v1`)
- CloudEvents 1.0 envelope + payload de negocio (alineado BIAN Control Record).
- Clave de partición: `accountId` o `customerId` para ordenamiento local.

- Idempotency-Key en header de comando y `eventId` UUID en eventos.
- Reintentos exponenciales + **DLQ** por topic; alerta SOAR si DLQ > umbral.
- Versionado: compatibilidad backward en vN; breaking → vN+1 + dual-publish temporal.

Eventos principales (referencia):

Evento	Productor	Consumidores	Payload (campos clave)
`party.customer.updated.v1`	Party MS / ACL	Fraude, OF, Canales	customerId, kycStatus, riskTier
`account.balance.changed.v1`	ACL CDC / Core Digital	Read Model, Riesgos	accountId, available, ledger, ts
`payment.order.created.v1`	Payment Order	Fraude async, Audit	orderId, amount, currency, debtor/creditor
`payment.execution.completed.v1`	Payment Execution	Notif, Ledger, OF	executionId, clearingRef, status
`payment.execution.failed.v1`	Payment Execution	Saga compensación, Notif	reasonCode, retryable
`fraud.decision.made.v1`	Fraud Evaluation	Payment, Case Mgmt	decision, score, rulesFired
`risk.limit.updated.v1`	Risk Services	Payment guard	customerId, product, limit, used
`consent.granted.v1` / `revoked.v1`	Open Finance	APIM policies, Audit	consentId, scopes, tppId, exp

---

5. Seguridad, cumplimiento y protección de datos

5.1 Controles ubicados en la arquitectura

Capa	Control	Ubicación exacta
Perímetro	WAF, DDoS, TLS 1.2+	Front Door / App Gateway
Edge API	OAuth2/OIDC, mTLS (TPP), rate limit, IP allowlist	**APIM**
Datos en tránsito	TLS end-to-end; mTLS service mesh interno (opcional)	AKS / Ingress
Datos en reposo	Cifrado AES-256; CMK en Key Vault	DBs, disks, Event Hubs
Secretos	Managed Identity; sin credenciales en repos	Todos los workloads
Privacidad	Tokenización PAN; enmascaramiento PII en logs	Payment, Observability pipelines
Fraude	Decisión síncrona en camino de pago	Fraud Evaluation ↔ Payment
AML/KYC	Screening en onboarding y pagos umbral	Risk + Party + Payment guards
Auditoría	Eventos inmutables + WORM storage evidencia	Event Store / Storage immutability
Segregación	RBAC Entra + namespaces AKS + private endpoints	IAM + red

5.2 Cumplimiento normativo (argumentado en el diseño)

El diseño no trata la normativa como checklist externo: los controles viven en **APIM, Consent Service, Party/SoR, Event Backbone, IAM y Observabilidad**. El marco primario de privacidad es la **Ley Orgánica de Protección de Datos**

Personales (LOPDP) del Ecuador (R.O. Suplemento 459, 26-may-2021) y su Reglamento; se complementa con Open Finance, PCI-DSS (si aplica tarjeta) y continuidad operativa bancaria.

5.2.1 Roles LOPDP en esta arquitectura

Rol LOPDP	Quién en el banco / ecosistema	Cómo se refleja en el diseño
Titular	Cliente (persona natural)	Ejercicio de derechos vía canales + API de Privacy Operations; identidad en CIAM
Responsable del tratamiento	El banco (dueño de la finalidad y del SoR)	Define finalidades, bases de legitimación, retención y DPIA; ownership por dominio BIAN/SoR
Encargado del tratamiento	Proveedores cloud, TPP bajo mandato, BPO	Contrato + instrucciones; acceso solo por API/mTLS; no SoR propio de datos del banco
Delegado de Protección de Datos (DPD)	Función de cumplimiento / privacy office	Consume evidencias: consent audit, access logs APIM, WORM, reportes de derechos

> **Transferencia a terceros (p. ej. Open Finance):** solo con base de legitimación + consentimiento informado cuando corresponda; el TPP recibe datos **acotados por scope/consent**, no acceso directo a cores. Revocación de consentimiento corta el acceso en APIM de forma inmediata (§8 / Consent Service).

5.2.2 Principios LOPDP (Art. 10) → componentes de la solución

Principio LOPDP	Cómo está presente en el diseño	Componente(s)
Juridicidad	Tratamiento solo con base legítima (contrato bancario, obligación legal, consentimiento OF, interés legítimo evaluado) documentada por finalidad	Party + Consent + catálogo de tratamientos
Lealtad / Transparencia	Avisos claros en canales; developer portal OF con finalidades; `problem+json` y headers de frescura de datos	Banca Web/Móvil, APIM portal
Finalidad	Cada API/evento declara *purpose*; Consent OF amarra scopes a fines; prohibido reutilizar payload de pago para marketing sin base	Consent Service, APIM policies, Schema Registry
Pertinencia y minimización	Field-level filtering en BFF/APIM; proyecciones CQRS con subset; Open Finance no expone PII innecesaria	BFF, APIM, Account Read Model
Proporcionalidad	Step-up MFA / fraude sync solo cuando monto/riesgo lo justifica; no scoring invasivo en consultas de saldo	Fraud Evaluation, IAM step-up
Confidencialidad	TLS, mTLS TPP, tokenización PAN, redaction PII en logs, RBAC/PIM	§5.1, SEC-03/04
Calidad y exactitud	SoR autoritativo por entidad; CDC + eventos corrigen read models; Party golden record	Matriz SoR, ACL CDC
Conservación	TTL por tipo de dato (consent, logs, audit WORM, ledger según obligación legal); jobs de purga sobre proyecciones	Privacy Operations + Data lifecycle
Seguridad de datos	Controles §5.1 / SEC-xx; cifrado en reposo con CMK; private endpoints; circuit breaker no degrada integridad del ledger	Key Vault, red, Saga

Responsabilidad proactiva y demostrada	Evidencia auditável: consent events, APIM access logs, traces OTel, decisiones fraude, DPIA ligadas a dominios	Observabilidad + Audit WORM
Aplicación favorable al titular	Ante duda en diseño de API OF o retención, se privilegia minimización y revocación efectiva	API Guild + Compliance review

5.2.3 Consentimiento (Art. 8 LOPDP) en la solución

El consentimiento debe ser **libre, específico, informado e inequívoco**, y **revocable** con el mismo nivel de facilidad con que se otorgó:

Requisito del consentimiento	Implementación en la arquitectura
Libre / inequívoco	Acción afirmativa en canal o flujo OF (no pre-ticked); registro de `consentId`, timestamp, versión de aviso
Específico	Scopes granulares (`accounts.read`, `payments.write`, scopes OF); un consentimiento ≠ "todo el banco"
Informado	Finalidad, destinatario (TPP), plazo y derechos visibles antes de autorizar
Demostrable	Eventos `consent.granted.v1` / `revoked.v1` + almacén Consent Service (SoR de consentimiento)
Revocable	Revocación en canal/OF → evento → **APIM deniega** tokens/scopes afectados sin redeploy; lag objetivo ≤ segundos
Distinto del contrato bancario	Operar cuenta (obligación contractual / legal) ≠ compartir datos con un TPP (consentimiento OF)

5.2.4 Derechos del titular → capacidades técnicas

Derecho del titular (LOPDP)	Capacidad en la plataforma	Notas de diseño
Información / transparencia	Avisos + portal; inventario de tratamientos por dominio	Catálogo mantenido por Domain Owners + DPD
Acceso	Orquestador de derechos consulta SoR + proyecciones y entrega expediente	No exponer DB interna; API Privacy + máscaras
Rectificación / actualización	Escritura solo en SoR Party / Account dueño; proyecciones vía eventos	Evita "corregir" solo el read model
Eliminación / cancelación	Flujo de borrado/anonimización con excepciones legales (ledger, AML, retención judicial)	Soft-delete + tombstone en bus; WORM audit de la decisión
Oposición / limitación	Flags de tratamiento en Party + políticas APIM	P. ej. no marketing; OF ya cubierto por revocación de consent
Portabilidad (cuando aplique)	Export estructurado (JSON) desde Party + Account views	Formato versionado; canal autenticado
Revocación de consentimiento	Consent Service + APIM (arriba)	Misma UX que el alta de consent

5.2.5 Otros marcos de cumplimiento (además de LOPDP)

Requisito	Cómo se integra en *esta* solución
Open Finance / Open Banking	Consent como SoR; mTLS + OAuth2 TPP; auditoría de cada acceso AISP/PISP; product plans en APIM

PCI-DSS (si hay datos de tarjeta)	PAN tokenizado; CDE segmentada; sin track data en logs; vendors compliant
Seguridad / continuidad bancaria	HA multi-AZ, DR multi-región, RTO/RPO y runbook (§6); integridad de ledger priorizada sobre disponibilidad degradada
Separación de funciones	Maker-checker en backoffice; PIM/JIT; sin privilegios permanentes (§8)
Encargados / cloud	Contratos de tratamiento; Managed Identity; datos en región acordada; DLP en egress

Argumento de cierre (tarea 3 del ejercicio): seguridad, LOPDP y cumplimiento no son anexos: el **Consent Service** materializa consentimiento y finalidad; **APIM/IAM** materializan confidencialidad y minimización en el borde; los **SoR + eventos** materializan exactitud y ejercicio de derechos; **Key Vault / cifrado / red** materializan seguridad; **Audit WORM + OTel** materializan responsabilidad proactiva demostrable ante la autoridad o el auditor.

5.3 Tabla de controles de seguridad (entregable)

ID	Amenaza	Control	Componente	Evidencia
SEC-01	Acceso no autorizado API	OAuth2 + scopes + mTLS TPP	APIM + IAM	Access logs APIM
SEC-02	Doble pago por reintento	Idempotency-Key + Inbox	Payment / Orchestrator	Métricas duplicados rechazados
SEC-03	Fuga PII en logs	Redaction processor OTel	Observability	Policy as code
SEC-04	Compromiso de secreto	Key Vault + rotation	Todos	Alertas acceso anómalo
SEC-05	Fraude en tiempo real	Scoring sync + case async	Fraud Evaluation	Decision audit trail
SEC-06	Manipulación de eventos	Firmas / schema registry + ACL produce-only	Event Backbone	Schema compatibility gates
SEC-07	Lateral movement	Private Link, NSG, zero-trust	Red cloud	Network watcher
SEC-08	Insider misuse	RBAC + PIM + audit	Entra ID	Access reviews

---

6. Alta disponibilidad y recuperación ante desastres

Estrategia **práctica anclada a este diseño** (no teórica genérica):

6.1 Objetivos

Flujo	RTO	RPO	Disponibilidad objetivo
Consulta saldo / movimientos (read model)	≤ 15 min	≤ 1 min	99.95%
Pagos / transferencias	≤ 30 min	≤ 1 min (outbox)	99.9%
Open Finance (lectura)	≤ 30 min	≤ 5 min	99.9%
Core Tradicional (dependencia)	Según contrato legacy	CDC lag monitoreado	Bulkhead + degradación

6.2 Matriz componente → HA → réplica DR

Componente	Estrategia HA (región A)	Réplica / DR (región B)
APIM	Premium multi-AZ	Instancia paired + Traffic Manager
AKS (Payment, Orchestrator, Fraud)	≥ 3 zonas, PDB, HPA	Cluster standby frío/tibio; images en ACR geo-rep
Event Backbone	RF≥3, min ISR	Mirror de topics críticos (async)
PostgreSQL por servicio	Zona-redundant / HA primary	Geo-replica async (RPO ≤ 1 min)
Redis (idempotency)	Réplica + AOF	Réplica secundaria o rebuild desde outbox
Account Read Model	Multi-réplica lectura	Rebuild desde bus + snapshot
Core Tradicional	Bulkhead; no controlamos HA	Dual-connectivity; runbook con ops legacy
Core Digital	Multi-AZ nativo	Failover del proveedor cloud del core
Key Vault / Entra	Geo-redundant	Misma identidad federada en B

6.3 HA (misma región)

- AKS: ≥ 3 zonas, PDBs, HPA por servicio crítico (Payment, Orchestrator, Fraud).
- APIM: unidad Premium multi-AZ / stv2.
- Kafka/Event Hubs: RF≥3, min ISR, topics críticos con retención ≥ 7 días.
- Redis idempotency: réplica + persistencia AOF/RDB.
- Cores: circuit breaker; si Core Legacy degradado → modo “solo consultas cacheadas” + cola de comandos con back-pressure.

6.4 DR — pasos numerados de failover (runbook)

1. **Detectar:** alerta RTO-clock (región A unhealthy > umbral) + confirmación humana/automatizada según severidad.
2. **Congelar escrituras no críticas** en A (feature flag global `writes.drain`).
3. **Drenar outbox** en A (publicar pendientes o marcar para replay).
4. **Promover stores** en B (Postgres replicas → primary; validar lag ≤ RPO).
5. **Validar Event Backbone** en B (consumer lag, ISR); habilitar mirror cutover.
6. **Validar CDC lag** Legacy → bus < umbral (p. ej. 30 s) antes de abrir débitos Legacy.
7. **Conmutar tráfico:** Traffic Manager / Front Door → APIM región B.
8. **Abrir lecturas** (read model); smoke test saldo + health.
9. **Abrir escrituras de pago** con rate limit reducido 15 min; monitorear sagas.
10. **Comunicar** estado a ops/negocio; abrir war-room hasta estabilización SLO.

Ejercicios de DR semestrales con métricas RTO reales. Payment Hub y esquemas externos: dual-connectivity pre-provisionada.

6.5 Sagas in-flight y pagos a mitad

Situación en failover	Tratamiento
Saga `STARTED` sin debit	Reintento idempotente en B con misma `Idempotency-Key`; o mark `FAILED` + cliente reintenta
Saga `DEBITED` (Legacy OK, credit pendiente)	**No abandonar:** worker de recuperación ejecuta Credit o `ReverseDebit`; reconciliación ledger vs saga store
Saga `COMPENSATING`	Continuar compensación hasta terminal; alertar si > 5 min

Outbox no publicado	Replay desde outbox table replicada / exportada antes del cutover
Cliente con timeout UX	Consulta `GET /payment-orders/{id}` (estado autoritativo); nunca segundo POST sin misma key

Regla de oro: **mejor rechazar nuevo pago (`503`) que duplicar posting**. La integridad de ledger prevalece sobre disponibilidad de escritura.

6.6 Degradación controlada (503 vs lectura cacheada)

Modo	Condición	Comportamiento canal
Lectura cacheada	Core Legacy lento / CB abierto; read model fresco (< TTL)	`200` con saldo proyectado + header `X-Data-Freshness: CACHED`
Lectura degradada	Read model stale > umbral	`200` con warning o `203` según producto; no inventar saldo
Escritura rechazada	Saga store / core destino indisponible	**`503`** + `Retry-After`; Problem+JSON `type=urn:bank:errors:dependency-unavailable`
Fraude timeout	Según política de monto	Fail-closed `503` montos altos; fail-open auditado montos bajos
Orden de sacrificio	Incidente prolongado	analytics → notif. no críticas → OF write → pagos no urgentes → **nunca** integridad ledger

---

7. Estrategia de integración multi-core

7.1 Modelo de convivencia

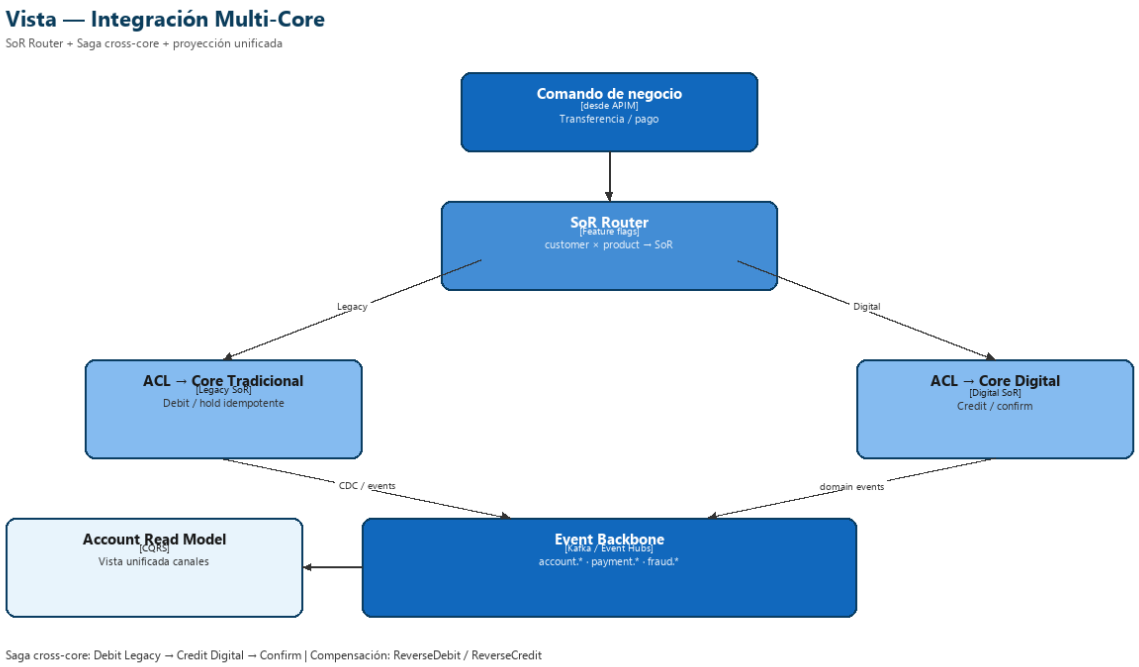


Figura 4. Integración Multi-Core

Reglas de ownership (SoR):

Dato / operación	SoR autoritativo (fase inicial)	Evolución

Cuenta nómina legacy	Core Tradicional	Migración Strangler por cohorte
Cuenta digital / neoproducto	Core Digital	Nativo
Saldo “vista canal”	Read Model (proyección)	Siempre derivado
Cliente (Party)	MDM / Party MS (golden record)	Cores reciben referencias
Consent Open Finance	Consent Service	Independiente de cores
Decisión fraude	Fraud System	Independiente

7.2 Escenarios multi-core

1. **Intra-core:** débito/crédito en el mismo SoR → transacción local del core + evento `payment.execution.completed`.
2. **Cross-core:** débito Legacy + crédito Digital → **Saga orquestada** (detalle en §3.5 / Figura 5):
   - Paso 1: hold/debit Legacy (`Idempotency-Key` step1, timeout 12 s)
   - Paso 2: credit Digital (timeout 8 s)
   - Paso 3: confirm / release + outbox
   - **Fallo del paso 2 tras debit Legacy:** estado `COMPENSATING` → `ReverseDebit` → `FAILED` + `payment.execution.failed.v1`
   - Compensación simétrica si el camino fuera Digital→Legacy: `ReverseCredit` cuando aplique
3. **Lectura unificada:** Channel BFF consulta **Account Read Model** alimentado por CDC Legacy + eventos Digital (no junta DBs).

7.3 Dónde se ve en el diseño

- Contenedor **Multi-Core Orchestrator** + componentes **SoR Router**, **Saga**, **ACL**, **Fraud Client**, **Payment Guard**.
- Eventos `account.*` desde ambos cores hacia el mismo bus semántico.
- Feature flags por `customerId`/`productCode` para cutover gradual.

---

8. Gestión de identidades y accesos (IAM)

8.1 Planos de identidad

Plano	Sujetos	Mecanismo
Clientes (CIAM)	Retail/empresas	OIDC + MFA adaptativo; tokens cortos; step-up para pagos
Empleados	Ops, riesgo, fraude	Entra ID + Conditional Access + PIM
Workloads	Microservicios	Managed Identity / SPIFFE; sin client secrets estáticos
TPP Open Finance	Fintechs	mTLS + OAuth2 client credentials / auth code + consent

8.2 Catálogo de scopes (referencia)

Scope	Audiencia	Uso
`accounts.read`	Cliente / BFF	Consulta saldos y movimientos (canal)
`payments.write`	Cliente / BFF	Crear órdenes de pago (requiere step-up si monto > umbral)
`openid` / `profile`	Cliente	Identidad CIAM

`openfinance.accounts`	TPP AISP	Lectura bajo consentimiento
`openfinance.payments`	TPP PISP	Iniciación de pago bajo consentimiento
`internal.payment.execute`	Workload Payment	Ejecución entre MS
`internal.fraud.decide`	Workload Payment → Fraud	Llamada síncrona fraude
`internal.account.post`	Workload Orchestrator → ACL	Débito/crédito vía puertos
`admin.consent.revoke`	Empleado / break-glass	Revocación operativa auditada

8.3 Autenticación, autorización y control entre componentes

1. **Autenticación de usuario:** CIAM emite access token; APIM valida JWT (issuer, aud, exp, nbf).
2. **Autorización de canal:** scopes (§8.2) + políticas ABAC (monto, device trust).
3. **Autorización servicio-servicio:** tokens de workload con audiences explícitas; **deny by default** en mesh/network policies.
4. **Token exchange / workload identity:** BFF presenta token de usuario; Payment obtiene token de workload (OAuth2 token exchange / On-Behalf-Of o Managed Identity → Entra) con `aud=payment-execution` y scope `internal.payment.execute`. Fraud Client usa MI propia con `aud=fraud-evaluation`. No se reenvía el token de usuario crudo a cores.
5. **Open Finance:** Consent Service valida consentimiento activo antes de liberar datos; APIM policy llama introspection/consent check.
6. **Break-glass:** cuentas de emergencia vaulted, alertadas, tiempo limitado.
7. **Propagación de identidad:** `sub` + `customerId` + `correlationId` en headers/baggage OTel (sin PII excesiva).

8.4 Principios

- Least privilege, separación maker-checker, rotación de claves, revisión trimestral de accesos TPP.

---

9. Estrategia de APIs internas y externas

9.1 Capas de API

Capa	Audiencia	Estilo	Estándar de mensajería
External Open Finance	TPP	REST + webhooks	Perfil regulatorio local + OAuth2; JSON; firmas
External Partner Payments	Esquemas/hubs	ISO 20022 (pacs/pain/camt) + APIs hub	XML/JSON ISO 20022
Channel Experience APIs	BFF web/móvil	REST agregadas	JSON canónico banco
Internal Domain APIs	MS internos	REST/gRPC	OpenAPI 3 / Protobuf; BIAN-aligned resources
Async Events	MS	Kafka topics	CloudEvents + Schema Registry (Avro/JSON Schema)

9.2 Esqueleto OpenAPI mínimo (Payment Order)

```
openapi: 3.0.3 info: { title: Payment Order API, version: 1.0.0 } paths: /internal/v1/payment-orders: post: operationId: createPaymentOrder parameters: - in: header name: Idempotency-Key required: true schema: { type: string, format: uuid } - in: header name: X-Correlation-Id required: true schema: { type: string } security: [{ oauth2: [payments.write, internal.payment.execute] }] responses: "201": description: Orden creada headers: Location: { schema: { type: string } } "409": description: Conflicto de idempotencia / estado content: application/problem+json: schema: { $ref:
```

```
"#/components/schemas/Problem" } "503": description: Dependencia no disponible headers: Retry-After: { schema: { type: integer } } content: application/problem+json: schema: { $ref: "#/components/schemas/Problem" } /internal/v1/payment-orders/{orderId}: get: operationId: getPaymentOrder parameters: - in: path name: orderId required: true schema: { type: string } responses: "200": { description: Estado autoritativo de la orden } components: schemas: Problem: type: object properties: type: { type: string, example: "urn:bank:errors:dependency-unavailable" } title: { type: string } status: { type: integer } detail: { type: string } correlationId: { type: string }
```

Headers transversales: `Authorization`, `Idempotency-Key` (writes), `X-Correlation-Id`, `X-Idempotency-Replayed: true` cuando se devuelve respuesta cacheada de la misma key.

9.3 Ejemplos de APIs

Interna — crear orden de pago

```
POST /internal/v1/payment-orders Idempotency-Key: 8f2c... X-Correlation-Id: 3fa85f64-... Authorization: Bearer <workload-or-user> Content-Type: application/json { "debtorAccountId": "ACC-LEG-001", "creditorAccountId": "ACC-DIG-889", "amount": { "value": "1500.00", "currency": "MXN" }, "purpose": "TRANSFER", "channel": "MOBILE" }
```

Externa Open Finance — cuentas (AISP)

```
GET /open-finance/v1/accounts Authorization: Bearer <consent-bound-token> x-fapi-interaction-id: 3fa85f64-...
```

9.4 Extracto ISO 20022 ↔ canónico

ISO 20022 (pacs.008 / pain.001)	Campo canónico	Notas
`CdtTrfTxInf/PmtId/EndToEndId`	`endToEndId`	Conservar para clearing/reconciliación
`IntrBkSttlmAmt` / `InstdAmt`	`amount.value` + `currency`	Canonical Mapper (§3.3)
`DbtrAcct/Id`	`debtorAccountId`	Tras resolución IBAN/CLABE → id interno
`CdtrAcct/Id`	`creditorAccountId`	Idem
`PmtTpInf` / purpose	`purpose`	Mapeo a catálogo banco
Status `ACCC`/`RJCT` (pacs.002)	`status` ejecución	Dispara eventos completed/failed

El **Payment Hub** habla ISO 20022 hacia esquemas; hacia dentro del banco solo circula el modelo canónico BIAN-aligned (ACL / Canonical Mapper).

9.5 Webhooks Open Finance

Aspecto	Diseño
Eventos	`consent.revoked`, `payment.initiation.updated` (ejemplos)
Firma	HMAC-SHA256 sobre body + timestamp; header `X-Bank-Signature`; secreto en Key Vault por TPP
Reintentos	Exponencial 1m, 5m, 30m, 2h, 12h; máx 24 h; luego DLQ operativa
Idempotencia	Header `X-Webhook-Id` único; TPP debe tratar duplicados como no-op
Seguridad	mTLS opcional; allowlist de URLs; no PII innecesaria en payload

9.6 Ejemplo de evento (CloudEvents)

```
{ "specversion": "1.0", "type": "payment.execution.completed.v1", "source": "urn:bank:payment-execution", "id":
"evt-9c2a...", "time": "2026-07-21T18:04:12Z", "subject": "PAY-77821", "datacontenttype": "application/json", "data": {
"executionId": "PAY-77821", "orderId": "ORD-441", "debtorAccountId": "ACC-LEG-001", "creditorAccountId": "ACC-DIG-889",
"amount": { "value": "1500.00", "currency": "MXN" }, "sorPath": ["LEGACY", "DIGITAL"], "clearingRef": null, "status":
"COMPLETED" } }
```


9.7 Políticas transversales

- Versionado URI (`/v1`) + deprecation window 180 días.
- Paginación cursor-based en listados.
- Problemas RFC 7807 (`application/problem+json`).
- Correlation ID obligatorio extremo a extremo.

---

10. Gobernanza de APIs y microservicios

10.1 Modelo operativo

Órgano	Responsabilidad
Architecture Review Board (ARB)	Límites de dominio BIAN, excepciones a patrones
API Guild	Estándares OpenAPI, linting, seguridad API
Platform Team	AKS, Event Backbone, Observability golden paths
Domain Owners	Backing SoR, SLOs, backlog del bounded context
Security & Compliance	Threat modeling, controles SEC-xx, auditorías

10.2 Ciclo de vida API

1. Diseño (OpenAPI-first) → 2. Review (breaking change detect) → 3. Publicación en APIM (producto/plan) → 4. Telemetría SLO → 5. Versionado/retiro.

10.3 Gobernanza de microservicios

- **Un equipo = un dominio** (ownership).
- Definition of Done: contratos, pruebas de contrato (Pact), dashboards SLO, runbooks, DLQ alertas.
- Prohibido: DB compartida, lógica en APIM, chatty sync chains > 3 hops (obligar async o BFF aggregation).
- Scorecards (ops level, security posture) trimestrales.

---

11. Plan de migración gradual (mínimo riesgo)

Fase 0 — Fundaciones (0–3 meses)

- APIM, IAM, Event Backbone, Observabilidad, ACL piloto, Account Read Model (CDC solo lectura).
- Sin cambio de SoR de escritura.

Fase 1 — Strangler de lecturas (3–6 meses)

- Canales leen saldos/movimientos desde Read Model.
- Core Legacy permanece SoR; se miden paridad y lag CDC.

Fase 2 — Productos digitales nativos (6–12 meses)

- Nuevos productos solo en Core Digital.
- Pagos intra-digital vía Orchestrator; fraude sync integrado.

Fase 3 — Cohortes cross-core (12–18 meses)

- Migración de clientes/productos por oleadas (feature flags).
- Sagas cross-core en producción con compensación probada en chaos tests.

Fase 4 — Retiro selectivo legacy (18–36 meses)

- Apagar módulos legacy ya sin tráfico; mantener adaptadores solo para residuales.
- Open Finance maduro sobre proyecciones + Consent.

Riesgo operativo mitigado por: dual-run, feature flags, idempotencia, reconciliación diaria ledger vs eventos, kill switches por producto.

11.1 Criterios go / no-go (por fase u oleada)

Criterio	Go	No-go
Paridad Read Model vs SoR	Error < 0.01% en muestra 7 días	Discrepancias sistemáticas de saldo
CDC lag p99	< 30 s	> 60 s sostenido
Saga success rate (cross-core)	≥ 99.5% excl. rechazos de negocio	Compensaciones > 1% sin causa clara
Fraud p99	≤ 80 ms	Breach sostenido sin mitigación
Chaos / DR drill	RTO medido ≤ objetivo	Fallo de runbook o pérdida de outbox
Seguridad	Controles SEC-xx evidentes; sin findings críticos abiertos	Findings críticos abiertos

11.2 Rollback

1. Desactivar feature flag de cohorte (`productCode`/`customerId` → SoR Legacy only).
2. Congelar nuevas sagas cross-core; drenar in-flight (§6.5).
3. Revertir tráfico de lectura al adapter Legacy directo si Read Model corrupto (kill switch).
4. Mantener Event Backbone encendido (no borrar topics); marcar oleada como `ROLLED_BACK` en registro de migración.
5. Post-mortem obligatorio antes de reintento de oleada.

---

12. Observabilidad y trazabilidad

12.1 Modelo de señales

Señal	Estándar	Uso
Traces	W3C Trace Context + OTel	Camino pago extremo a extremo (BFF→APIM→Saga→Core→Event)
Métricas	RED/USE + SLIs	Latencia p99 fraude, tasa DLQ, CDC lag, success rate pagos
Logs	Estructurados JSON	Sin PII cruda; `correlationId`, `customerId` hasheado si aplica

Audit	Eventos inmutables	Quién/qué/cuándo para regulador

12.2 Trazabilidad de negocio

- `correlationId` generado en canal/APIM.
- `paymentOrderId` / `executionId` como business keys en spans y eventos.
- Mapa de topología: service catalog + trace-to-BIAN-domain tagging (`bian.domain=PaymentExecution`).

12.3 Alertas accionables

- CDC lag > 30s, DLQ > 0 sostenido, error rate saga compensación, circuit breaker abierto Core Legacy, consentimiento denegado anómalo TPP.

12.4 SLOs numéricos (referencia operativa)

SLO	Objetivo	Ventana	Notas
Disponibilidad API pagos (APIM→Payment)	99.9%	30 días	Excluye rechazos 4xx de negocio
Latencia creación orden (p99)	≤ 350 ms	7 días	Hasta `ACCEPTED_FOR_EXECUTION` (§3.4)
Latencia consulta saldo read model (p99)	≤ 80 ms	7 días	Sin fallback SoR
Fraud sync (p99)	≤ 80 ms	7 días	Burn rate alerta en 1 h
CDC lag Legacy (p99)	≤ 30 s	24 h	Bloquea go de oleada si se viola
Entrega evento outbox (p99)	≤ 5 s	24 h	Desde commit local a bus
Tasa compensación saga indebida	< 0.2%	7 días	Excluye fallos de negocio esperados
DLQ profundidad	0 sostenido > 15 min	continuo	Página a Platform + Domain Owner

---

13. Anexos

A. Matriz Systems of Record

Entidad	SoR	Réplicas / proyecciones	Notas
Customer (Party)	Party MS / MDM	Cores (ref), Canales cache	Golden record
Account master	Core dueño del producto	Read Model	Router conoce dueño
Ledger entries	Core dueño	Eventos + data lake	No escribir desde MS satélite

Payment order	Payment Domain DB	—	Ciclo de vida propio
Fraud decision	Fraud System	Event copy audit	Autoridad de decisión
Risk limits	Risk System	Cache en Payment guard	TTL corto
Consent	Consent Service	APIM cache policy	Revocació inmediata
Product catalog	Product Directory	CDN/cache canales	

B. Evita anti-patrones del enunciado

Anti-patrón evitado	Mecanismo en la propuesta
Diagramas genéricos no bancarios	Flujos de pago, Open Finance, multi-core, fraude RT
MS sin ownership	Matriz BIAN + SoR
DB compartida	Database per service
Cadenas síncronas largas	Max hops + eventos + CQRS
Lógica en API Gateway	APIM solo políticas
BIAN cosmético	Dominios, control records, naming
Vendor lock sin justificación	Azure de referencia + equivalencias AWS
Seguridad genérica	Tabla SEC-xx mapeada a componentes

C. Glosario breve

- **SoR:** System of Record (fuente autoritativa).
- **ACL:** Anti-Corruption Layer.
- **BIAN:** Banking Industry Architecture Network.
- **TPP:** Third Party Provider (Open Finance).
- **DLQ:** Dead-Letter Queue.

Conclusión

La arquitectura propuesta moderniza el banco mediante un **plano de integración multi-core event-driven**, contratos BIAN explícitos, seguridad embebida por capa, HA/DR runbookable y una migración Strangler Fig con go/no-go medibles. Los diagramas (Context, Container, Component), la saga cross-core y los anexos (SoR, APIs, eventos, controles) constituyen el núcleo del entregable para decisión ejecutiva y ejecución técnica.